

Improving efficiency of implicit Markov chain state classification

Andrew S. Miner

Shuxing Cheng

Department of Computer Science

Iowa State University

E-mail: {asminer, shuxing}@iastate.edu

Abstract

Current efficient symbolic methods to classify the states of a Markov chain into transient and recurrent classes use an iterative approach, where each iteration begins by selecting a “seed” state. In this paper we present heuristics to reduce the number of iterations required. Our core contribution is the use of shortest distance information to select the seed state. Our approach uses multiway decision diagrams to represent sets of states and edge-valued decision diagrams to represent distance information. Experimental results indicate that the distance heuristics can be quite effective, often minimizing the required number of iterations.

1. Introduction

Markov Chains are a well-established and important tool to assess the performance and dependability of computer and communication systems. An important step in many types of Markov chain analysis is the classification of states into recurrent classes and the class of transient states [15, 22]. For instance, the stationary distribution of a Markov chain is computed by analyzing the recurrent classes in isolation, and determining the probability that the Markov chain reaches each recurrent class. Knowledge of recurrent classes is useful for both exact numerical solution of the Markov chain and for discrete-event simulations to compute Markov reward measures based on the Markov chain. Recurrent classes can be discovered by first constructing the strongly-connected components (SCCs) of the Markov chain, e.g., using traditional algorithms [1]. Once the SCCs have been determined, the Markov chain states can be classified: each SCC with no transition to another SCC is a recurrent class; all others form the transient class.

A major hurdle is that the number of states in a Markov chain can be extremely large for practical systems. This state-space explosion problem effectively limits the applicability of explicit techniques, such as the traditional SCC computation, which require visiting and representing each

state of the Markov chain explicitly. As a result, many researchers have turned instead to implicit (also known as symbolic) techniques, which exploit structural characteristics of the system. Such techniques typically utilize decision diagrams, in particular binary decision diagrams (BDDs) [4] and multiway decision diagrams (MDDs) [14], to represent sets of Markov chain states [7, 24]. Compact, implicit representations for the transition matrix of the Markov chain include Kronecker representations [5, 11, 21], multi-terminal BDDs (also known as algebraic decision diagrams) [12, 13], or matrix diagrams [8].

Classification of Markov chain states can be done implicitly in several ways. Early work was based on transitive closure computation [12, 18, 23]; however, this tends to be computationally expensive. Alternative methods use an iterative approach, in which an initial seed state is selected: [24] constructs the complete forward and backward reachability sets (starting from the seed state) and determines which of those states are transient and recurrent, while [3] computes SCCs using a lockstep search to avoid complete exploration of either the forward or backward set. Iterations continue until all states have been classified. Note that the choice of seed state for a particular iteration may affect the computational cost to perform the iteration, and may affect the total number of iterations required.

In this paper, we investigate improvements to the above iterative methods for Markov chain state classification. We first use more efficient algorithms for constructing reachability sets for asynchronous systems, to reduce the computation time required per iteration. Our primary contribution is to consider heuristics based on distance information to select the seed states, rather than using random selection, in an effort to decrease the number of iterations required.

The remainder of the paper is organized as follows. Section 2 provides a brief overview of background and related work, including MDDs, reachability set generation, and iterative classification algorithms. Section 3 describes our distance-based heuristics for reducing the number of iterations. Experimental results for a variety of models are given in Section 4. Finally, Section 5 concludes the work.

2. Background and related work

We consider finite Markov chains constructed from a high-level, structured formalism with K components. Each component k consists of local states $\mathcal{S}_k = \{0, \dots, n_{k-1}\}$, and the set of *potential* states of the overall model is $\hat{\mathcal{S}} = \mathcal{S}_K \times \dots \times \mathcal{S}_1$. Thus, a Markov chain state can be written as $\mathbf{i} = (i_K, \dots, i_1)$. However, depending on the initial distribution, normally only a portion of the potential states can be reached; this set of *actual* states $\mathcal{S}$ (defined more formally below) is the state space of the Markov chain.

For the purposes of state classification, the precise value of the probability (or rate) of transition from one state to another is irrelevant; it matters only if the probability is zero or not. To this end, for the remainder of the paper we describe a Markov chain in terms of a next-state function $\mathcal{N} : \hat{\mathcal{S}} \rightarrow 2^{\hat{\mathcal{S}}}$, where $\mathcal{N}(\mathbf{i})$ specifies the set of states in the Markov chain that can be reached from state $\mathbf{i} \in \hat{\mathcal{S}}$ in one step; i.e., the states to which there is a nonzero probability (or rate) of transition from state $\mathbf{i}$. For notational convenience, we extend the next-state function to apply also to sets of states: $\mathcal{N}(\mathcal{X}) = \bigcup_{\mathbf{i} \in \mathcal{X}} \mathcal{N}(\mathbf{i})$.

One-step *forward* reachability from a state $\mathbf{i}$ is determined by $\mathcal{N}(\mathbf{i})$; this can be reversed to obtain one-step *backward* reachability $\mathcal{P}(\mathbf{i})$ with $\mathcal{P} = \mathcal{N}^{-1}$. We also use the notion of n -step forward reachability from a state or set of states: $\mathcal{N}^{n+1}(\mathbf{i}) = \mathcal{N}(\mathcal{N}^n(\mathbf{i}))$ with $\mathcal{N}^0(\mathbf{i}) = \{\mathbf{i}\}$. We can define n -step backward reachability $\mathcal{P}^n$ in a similar way. Forward reachability in any number of steps (including zero) can be defined as $\mathcal{N}^*(\mathbf{i}) = \bigcup_{n \in \mathbb{Z}_{\geq 0}} \mathcal{N}^n(\mathbf{i})$ with a similar definition for backward reachability $\mathcal{P}^*$. Note that $\mathcal{N}^n$, $\mathcal{P}^n$, $\mathcal{N}^*$ and $\mathcal{P}^*$ are all subsets of $\hat{\mathcal{S}}$, which we assume to be finite. In particular, if $\mathcal{S}^0$ is the set of states with nonzero probability at time 0 (i.e., the initial states), then the actual state space is $\mathcal{S} = \mathcal{N}^*(\mathcal{S}^0)$.

2.1. Implicit Markov chain representations

Several implicit representations for the transition matrix of a Markov chain have been proposed, including Kronecker representations [5, 11, 21], multi-terminal BDDs [12, 13], and matrix diagrams [8]. We require a representation that allows for the use of the saturation algorithm [7] to construct reachability sets. The techniques described in [7] make use of a Kronecker representation (described below); however, these approaches have already been adapted for use with matrix diagrams [19] and the same could probably be done for multi-terminal BDDs.

We assume that the high-level model describing the Markov chain consists of a set of events $\mathcal{E}$; these usually correspond to features of the model (e.g., departure from a queue, a Petri net transition, etc.). To use a Kronecker representation, it is necessary to express the next-state function

$\mathcal{N}$ as the union of several next-state functions, one for each event, expressible as

$$\mathcal{N}^e = \mathcal{N}_K^e \times \dots \times \mathcal{N}_1^e$$

where function $\mathcal{N}_k^e : \mathcal{S}_k \rightarrow 2^{\mathcal{S}_k}$ describes the effect of event e on component k , and must depend only on component k . If each function $\mathcal{N}_k^e$ is represented as a boolean matrix $\mathbf{N}_k^e$, such that $\mathbf{N}_k^e[i_k, j_k] = 1$ iff $j_k \in \mathcal{N}_k^e(i_k)$, then the boolean matrix representing function $\mathcal{N}^e$ is given by $\mathbf{N}^e = \mathbf{N}_K^e \otimes \dots \otimes \mathbf{N}_1^e$, and the boolean matrix representing the complete next-state function is $\mathbf{N} = \sum_{e \in \mathcal{E}} \mathbf{N}^e$. The actual state space can be constructed from such a representation, either explicitly [16] or implicitly (described below). Note that, using a Kronecker representation, inverting the next-state function $\mathcal{N}$ to obtain $\mathcal{P}$ can be done by simply transposing each matrix $\mathbf{N}_k^e$.

2.2. Encoding sets of states with MDDs

An MDD [14] is a directed, acyclic graph consisting of m *terminal* nodes, each labeled from integers $\{0, \dots, m-1\}$, and *non-terminal* nodes p , each labeled with a variable v_k and containing a vector of n_k pointers to other nodes, denoted $p.\text{down}$. We use the notion of *levels*: we say terminal nodes belong to level 0 and non-terminal nodes with variable v_k belong to level k . The level of a node p is denoted $\text{Top}(p)$. Two level- k nodes p and p' are said to be *duplicates* if $p.\text{down} = p'.\text{down}$. An MDD is *quasi-reduced* if it contains no duplicates, it contains a level- K node, and all pointers from a level- k node are to nodes at level $k-1$. Quasi-reduced MDDs are known to be canonical representations [14, 17] for a fixed ordering of levels; it is also known that the number of MDD nodes required is sensitive to the level order [4]. For the remainder of the paper, we assume all MDDs are quasi-reduced.

An MDD can be used to represent any subset of $\hat{\mathcal{S}}$. A given state $\mathbf{i} = (i_K, \dots, i_1)$ is contained in a set $\mathcal{X} \subseteq \hat{\mathcal{S}}$ represented by a K -level MDD iff the path through the MDD, starting at the level- K node, following downward pointer i_k at level k , reaches terminal node 1. Operations such as union, difference, and intersection can be performed directly on their MDD representations [20]; the complexity of these operations depends on the number of MDD nodes, rather than the cardinalities of the encoded sets.

An example MDD is shown in Figure 1(a), where $K = 3$, and $\mathcal{S}_3 = \mathcal{S}_2 = \mathcal{S}_1 = \{0, 1, 2\}$. Pointers to terminal node 0 are omitted for clarity. The MDD encodes the set $\mathcal{X} \subset \mathcal{S}_3 \times \mathcal{S}_2 \times \mathcal{S}_1$ where at most one component is in local state 2. Note for instance that state $(1, 0, 2)$ belongs to $\mathcal{X}$, because the corresponding path in the MDD reaches terminal node 1; conversely, $(2, 2, 0)$ does not belong to $\mathcal{X}$.

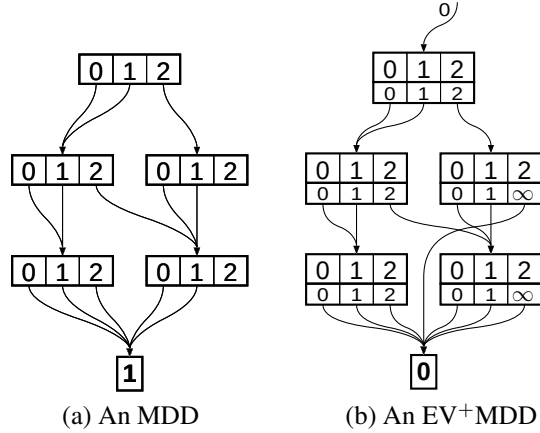


Figure 1. Examples of MDDs and EV⁺MDDs

2.3. EV⁺MDDs and distance information

We will also utilize edge-valued MDDs (EV⁺MDDs) as described in [9]. EV⁺MDDs are similar to MDDs, but assign an edge value (either infinity or a natural) to each downward pointer. We denote the edge values for node p as the vector $p.\text{value} \in (\mathbb{N} \cup \{\infty\})^{n_k}$, with $p.\text{value}[i]$ as the edge value corresponding to $p.\text{down}[i]$. EV⁺MDDs require only a single terminal node, labeled with 0. An EV⁺MDD can encode a function $f : \hat{\mathcal{S}} \rightarrow \mathbb{N} \cup \{\infty\}$, where $f(\mathbf{i})$ is determined by following the path for $\mathbf{i}$ through the EV⁺MDD and summing the edge values encountered until terminal node 0 is reached. Equivalently, an EV⁺MDD can be viewed as a representation for a subset of $\hat{\mathcal{S}}$ where each element of the set has an associated natural value: $\mathbf{i}$ is not contained in the subset if $f(\mathbf{i}) = \infty$, otherwise $\mathbf{i}$ is contained in the subset and has associated value $f(\mathbf{i})$.

Reduction rules for EV⁺MDDs, discussed in [9], are similar to those for MDDs: two level- k nodes p and q are duplicates if $p.\text{down} = q.\text{down}$ and $p.\text{value} = q.\text{value}$. However, additional rules are required to obtain a canonical form. Since any information after an infinite edge value is irrelevant, by convention such edges are always directly to terminal node 0. Additionally, each node must contain an edge value of zero. Useful operators for EV⁺MDDs include UnionMin and IntersectionMax, described in [9], which return new EV⁺MDDs encoding the minimum and maximum of the functions encoded by the operands, respectively. An example EV⁺MDD is shown in Figure 1(b), which encodes function f where $f(\mathbf{i}) = i_3 + i_2 + i_1$ if $\mathbf{i} \in \mathcal{X}$ (with $\mathcal{X}$ represented by the MDD in Figure 1(a)), $f(\mathbf{i}) = \infty$ otherwise.

As discussed in [9], EV⁺MDDs are well-suited for encoding the distance function d_i , which indicates the minimum number of forward steps required to reach each state

$$d_i(\mathbf{j}) = \begin{cases} \min\{n : \mathbf{j} \in \mathcal{N}^n(\mathbf{i})\} & \text{if } \mathbf{j} \in \mathcal{N}^*(\mathbf{i}) \\ \infty & \text{if } \mathbf{j} \notin \mathcal{N}^*(\mathbf{i}) \end{cases}$$

Forward(in: MDD $\mathcal{X}$, Markov chain $\mathcal{N}$; out: MDD $\mathcal{S}$)

• Computes $\mathcal{S} = \mathcal{N}^*(\mathcal{X})$

1: $\mathcal{F} \leftarrow \mathcal{X}$;

2: $\mathcal{S} \leftarrow \mathcal{X}$;

3: **while** $\mathcal{F} \neq \emptyset$ **do**

4: $\mathcal{F} \leftarrow \mathcal{N}(\mathcal{F}) \setminus \mathcal{S}$;

5: $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{F}$;

6: **end while**

Figure 2. Fixed-point computation for $\mathcal{N}^*(\mathcal{X})$

from an initial state $\mathbf{i}$ (or set of initial states). Note that the EV⁺MDD in Figure 1(b) encodes the distances from the initial state $(0, 0, 0)$, where each local component may change from local state 0 to 1 and from 1 to 2 if no other component is in state 2.

2.4. Saturation for reachability

The problem of constructing $\mathcal{N}^*(\mathbf{i})$, also known as reachability set generation, is well-studied. The “traditional” approach to construct the reachability set using implicit methods (i.e., those based on BDDs and MDDs) is to use a fixed-point computation [10, 24], such as the one shown in Figure 2. Algorithm Forward constructs the set $\mathcal{N}^*(\mathcal{X})$ for a given set of states $\mathcal{X}$. At iteration n of the fixed-point computation, the set of states with distance equal to n is constructed (set $\mathcal{F}$) and added to $\mathcal{S}$. After the states with largest distance from the initial state have been added to $\mathcal{S}$, the set $\mathcal{F}$ will become empty and the algorithm terminates. A similar algorithm can be used to construct $\mathcal{P}^*(\mathcal{X})$.

For asynchronous systems, which contain several identity matrices in the Kronecker representation, the saturation algorithm [7] is able to generate extremely large sets $\mathcal{N}^*(\mathbf{i})$ and $\mathcal{P}^*(\mathbf{i})$ very efficiently, by exploiting the fact that many events affect only a few components. Experimental studies have shown that using the saturation algorithm to construct large sets is orders of magnitude faster than the traditional fixed-point computation (i.e., algorithms similar to Figure 2) for asynchronous systems. Furthermore, a variant of saturation [9] can be used to efficiently construct the EV⁺MDD representation of a distance function d_i .

Use of the saturation algorithm requires partitioning the next-state function $\mathcal{N}$ into $\mathcal{N}_K \cup \dots \cup \mathcal{N}_1$, where $\mathcal{N}_k$ contains transitions that do not affect components K through $k+1$. If a Kronecker representation is used, this partitioning is based upon events: $\mathcal{N}_k$ contains all events that do not affect components K through $k+1$ [7]. Alternatively, if a matrix diagram representation is used, the partitioning can be performed by examining the matrix diagram structure [19]. During reachability set generation, the next-state functions are applied repeatedly and exhaustively to MDD nodes; in

```

ReachClassify(in: MDD  $\mathcal{S}$ , Markov chain  $\mathcal{N}$ )
1:  $\mathcal{P} \leftarrow \mathcal{N}^{-1}$ ;  $\mathcal{S}' \leftarrow \mathcal{S}$ ;
2:  $C \leftarrow 0$ ;  $\mathcal{T} \leftarrow \emptyset$ ;
3: while  $\mathcal{S}' \neq \emptyset$  do
4:    $\mathbf{i} \leftarrow \text{SelectRandom}(\mathcal{S}')$ ;
5:    $\mathcal{F} \leftarrow \mathcal{N}^*(\mathbf{i})$ ;
6:    $\mathcal{B} \leftarrow \mathcal{P}^*(\mathbf{i})$ ;
7:    $\mathcal{S}' \leftarrow \mathcal{S}' \setminus \mathcal{B}$ ;
8:   if  $\mathcal{F} \subseteq \mathcal{B}$  then                                •  $\mathbf{i}$  is recurrent
9:      $C \leftarrow C + 1$ ;
10:     $\mathcal{R}_C \leftarrow \mathcal{F}$ ;
11:     $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B} \setminus \mathcal{F}$ ;
12:   else                                              •  $\mathbf{i}$  is transient
13:      $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B}$ ;
14:   end if
15: end while

```

Figure 3. Reachability-based classification.

particular, any newly-created level- k MDD node is “saturated” by exhaustively applying $\mathcal{N}_k$.

2.5. Markov chain state classification

Recall [15, 22] that each state $\mathbf{i}$ of a finite Markov chain can be classified as either *transient*, if it is possible to leave state $\mathbf{i}$ and never return, or *recurrent* otherwise. Furthermore, recurrent states can be partitioned into classes, where two recurrent states $\mathbf{i}$ and $\mathbf{j}$ are within the same class if it is possible to eventually reach $\mathbf{j}$ from $\mathbf{i}$ and vice-versa. Given a Markov chain with actual states $\mathcal{S} = \mathcal{N}^*(S^0)$, the problem we consider is how to classify the states $\mathcal{S}$ into recurrent classes and the set of transient states. We denote the set of transient states as $\mathcal{T}$, and the recurrent classes as $\mathcal{R}_1, \dots, \mathcal{R}_C$, where C is the number of recurrent classes. Note that $\mathcal{T} \cup \mathcal{R}_1 \cup \dots \cup \mathcal{R}_C = \mathcal{S}$.

One implicit method to classify states is due to Xie and Beereel [24], which we refer to as the reachability method. The method is based on the following properties:

- A state $\mathbf{i} \in \mathcal{S}$ is recurrent if $\mathcal{N}^*(\mathbf{i}) \subseteq \mathcal{P}^*(\mathbf{i})$, otherwise it is transient.
- If state $\mathbf{i}$ is recurrent, then $\mathcal{N}^*(\mathbf{i})$ is a recurrent class, and states $\mathcal{P}^*(\mathbf{i}) \setminus \mathcal{N}^*(\mathbf{i})$ are transient.
- If state $\mathbf{i}$ is transient, then states in $\mathcal{P}^*(\mathbf{i})$ are transient.

These properties lead to the classification algorithm shown in Figure 3, which appears in [24]. In the algorithm, $\mathcal{S}'$ denotes the set of states that are still unclassified.

There is an issue with the above method when a Kronecker representation is used for the Markov chain. If the set $\mathcal{S}$ is a proper subset of $\hat{\mathcal{S}}$, then a Kronecker representation of the Markov chain might contain transitions due

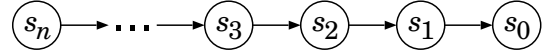


Figure 4. A linear Markov chain

to unreachable states (i.e., states in $\hat{\mathcal{S}} \setminus \mathcal{S}$), and it is possible that an unreachable state can reach a reachable state [5]: $\mathcal{N}(\hat{\mathcal{S}} \setminus \mathcal{S}) \cap \mathcal{S} \neq \emptyset$. This implies that the set $\mathcal{S}$ might not be closed under backward reachability: $\mathcal{P}^*(\mathcal{S}) \setminus \mathcal{S} \neq \emptyset$. Thus, as shown, algorithm ReachClassify may generate a set $\mathcal{T}$ containing unreachable states. This can be corrected either by removing unreachable states from $\mathcal{P}^*(\mathbf{i})$ at each iteration (using $\mathcal{B} \leftarrow \mathcal{P}^*(\mathbf{i}) \cap \mathcal{S}$), or by removing unreachable states from $\mathcal{T}$ at the end of the classification algorithm.

Note that each iteration of ReachClassify randomly selects an unclassified state and generates the entire forward and backward reachability sets; during each iteration, states $\mathcal{B} \cap \mathcal{S}'$ are classified. The time required to construct the forward and backward sets will depend on the seed state. The choice of seed state will also affect the total number of iterations required. The minimum number of iterations occurs when every seed state selected is recurrent, and there will be exactly C iterations. Random selection can perform poorly when the number of transient states is much larger than the number of recurrent states, since it is unlikely that the seed state will be recurrent. In the worst case, $\mathcal{B} \cap \mathcal{S}'$ contains only the seed state; if this occurs every time, then there will be exactly $|\mathcal{T}| + C$ iterations. For example, for the linear Markov chain shown in Figure 4, the best case occurs when state s_0 is selected, as this will classify all states in a single iteration; the worst case occurs when s_n is selected, then s_{n-1} , and so on, which will require $n + 1$ iterations.

Another technique to classify Markov chain states is based on lockstep search [3]. In contrast to [24], which constructs only the terminal SCCs, the algorithm in [3] is designed to construct *all* SCCs. The number of iterations required by the lockstep search algorithm is exactly equal to the number of SCCs in the Markov chain. Given the set of SCCs of the Markov chain, the recurrent classes can be determined by checking each SCC $\mathcal{C}$ to see if it is terminal: if $\mathcal{N}(\mathcal{C}) \subseteq \mathcal{C}$, then $\mathcal{C}$ is a recurrent class; otherwise, $\mathcal{C}$ is added to the set of transient states. Note that the number of SCCs can be much larger than the number of recurrent classes: for instance, for the Markov chain shown in Figure 4, every state is its own SCC. For this reason, we do not consider this approach any further here.

3. Selecting seed states

The number of iterations required by algorithm ReachClassify depends on the seed states selected. As described earlier, the minimum number of iterations achievable is

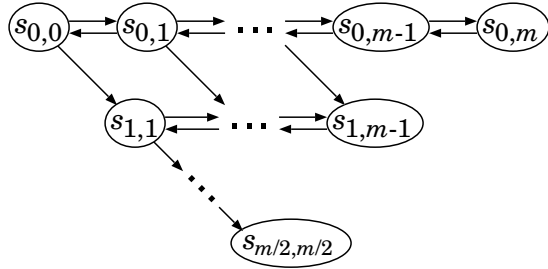


Figure 5. A “triangular” Markov chain

equal to C , the number of recurrent classes, when the selected seed state is always recurrent. Thus, procedure `SelectRandom` should always try to select a recurrent state; failing that, a seed state that causes a large number of transient states to be classified should be selected.

We propose two simple heuristics for selecting seed states based on distance information:

1. Always select an unclassified state with furthest distance from the set of initial states $\mathcal{S}^0$.
2. Always select an unclassified state with furthest distance from a set of states $\mathcal{X}$. Initially, $\mathcal{X}$ equals the set of initial states $\mathcal{S}^0$. If a transient state was selected, reset $\mathcal{X}$ to the current set of classified states.

The above rules, while simple, attempt to select a seed state with a large backward reachability set based solely upon shortest distance information. Note that, if the first heuristic works perfectly and always selects a recurrent state, then the second heuristic will be equivalent. Otherwise, whenever a transient state is selected, the second heuristic attempts to improve itself by updating its distance function.

Note that both heuristics will select seed state s_0 in the Markov chain shown in Figure 4, assuming $\mathcal{S}^0 = \{s_n\}$. An example where the second heuristic performs better than the first is shown in Figure 5. If the initial state is $\mathcal{S}^0 = \{s_{0,0}\}$, then state $s_{i,j}$ has shortest distance j from $\mathcal{S}^0$. Using the first heuristic, transient state $s_{0,m}$ will be selected first, and states $s_{0,j}$ will be classified as transient; then transient state $s_{1,m-1}$ will be selected, and so on, until finally selecting recurrent state $s_{m/2,m/2}$. Thus, heuristic (1) requires $m/2$ iterations. Using the second heuristic, after transient state $s_{0,m}$ is selected, the distance function is reconstructed, and state $s_{i,j}$ will have shortest distance i from the classified states (in this case, all states $s_{0,j}$). Recurrent state $s_{m/2,m/2}$ will be selected for the second iteration.

Efficient implementation of the distance-based heuristics is possible with EV^+MDDs , which can be used to encode distance information. Selection of a state with furthest distance, given a distance function represented as an EV^+MDD , can be done using procedure `SelectFurthest`,

```

MaxDistance(in:  $\text{EV}^+\text{MDD } p$ ; out:  $i, d$ )
1: if  $p = 0$  then
2:    $i \leftarrow 0$ ;  $d \leftarrow 0$ ;
3: else if  $\exists i', d' : (p, i', d') \in \text{cache}$  then
4:    $i \leftarrow i'$ ;  $d \leftarrow d'$ ;
5: else
6:    $d \leftarrow -1$ ;  $k \leftarrow \text{Top}(p)$ ;
7:   for each  $i_k \in \mathcal{S}_k$  do
8:      $\text{MaxDistance}(p.\text{down}[i_k], i', d')$ ;
9:     if  $\infty > p.\text{value}[i_k] + d' > d$  then
10:       $d \leftarrow p.\text{value}[i_k] + d'$ ;
11:       $i \leftarrow i_k$ ;
12:   end if
13: end for
14:  $\text{cache} \leftarrow \text{cache} \cup \{(p, i, d)\}$ ;
15: end if

SelectFurthest(in:  $\text{EV}^+\text{MDD } p$ )
1: while  $p \neq 0$  do
2:    $k \leftarrow \text{Top}(p)$ ;
3:    $\text{MaxDistance}(p, i_k, d)$ ;
4:    $p \leftarrow p.\text{down}[i_k]$ ;
5: end while
6: return  $(i_K, \dots, i_1)$ ;

```

Figure 6. Distance-based seed selection

shown in Figure 6. The procedure selects a path through the EV^+MDD so that the sum of the edge values is as large as possible, but still finite. This path is selected using recursive procedure `MaxDistance`, also shown in Figure 6, which determines for a given EV^+MDD node p the maximum finite value of the function encoded by p , and a downward pointer that must be followed to obtain this value. Note that `SelectFurthest` finds the furthest state by simply traversing the EV^+MDD according to the downward pointer that should be followed, as determined by `MaxDistance`. Similar to other decision diagram algorithms [4], procedure `MaxDistance` utilizes a cache to prevent duplication of computation (lines 3 and 14): once the maximum value and downward pointer have been determined for a particular node, this information is saved and used immediately for subsequent identical calls to `MaxDistance`.

There may be several states with largest distance in the EV^+MDD . In this case, our implementation of `SelectFurthest` uses the first discovered state with largest distance. However, other policies could be implemented, such as random selection or other heuristics. Note that, in the case where all reachable states are initial states ($\mathcal{S}^0 = \mathcal{S}$), all distances will be zero, and the distance heuristics will be useless. For most practical applications with extremely large Markov chains, however, the set of initial states $\mathcal{S}^0$ is a small fraction of the reachable states $\mathcal{S}$.

A classification algorithm based on heuristic (1) is shown

```

Distance1Classify(in: MDD  $\mathcal{S}$ , EV+MDD  $\mathcal{D}$ , MC  $\mathcal{N}$ )
1:  $\mathcal{P} \leftarrow \mathcal{N}^{-1}$ ;  $\mathcal{S}' \leftarrow \mathcal{S}$ ;
2:  $C \leftarrow 0$ ;  $\mathcal{T} \leftarrow \emptyset$ ;
3: while  $\mathcal{S}' \neq \emptyset$  do
4:    $\mathbf{i} \leftarrow \text{SelectFurthest}(\mathcal{D})$ ;           • changed line
5:    $\mathcal{F} \leftarrow \mathcal{N}^*(\mathbf{i})$ ;
6:    $\mathcal{B} \leftarrow \mathcal{P}^*(\mathbf{i}) \cap \mathcal{S}$ ;           • changed for Kronecker
7:    $\mathcal{S}' \leftarrow \mathcal{S}' \setminus \mathcal{B}$ ;
8:   if  $\mathcal{F} \subseteq \mathcal{B}$  then
9:      $C \leftarrow C + 1$ ;
10:     $\mathcal{R}_C \leftarrow \mathcal{F}$ ;
11:     $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B} \setminus \mathcal{F}$ ;
12:   else
13:      $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B}$ ;
14:   end if
15:    $\mathcal{D} \leftarrow \text{IntersectionMax}(\mathcal{D}, \mathcal{B} \cdot \infty)$ ;   • new line
16: end while

```

```

Distance2Classify(in: MDD  $\mathcal{S}$ , EV+MDD  $\mathcal{D}$ , MC  $\mathcal{N}$ )
1:  $\mathcal{P} \leftarrow \mathcal{N}^{-1}$ ;  $\mathcal{S}' \leftarrow \mathcal{S}$ ;
2:  $C \leftarrow 0$ ;  $\mathcal{T} \leftarrow \emptyset$ ;
3: while  $\mathcal{S}' \neq \emptyset$  do
4:    $\mathbf{i} \leftarrow \text{SelectFurthest}(\mathcal{D})$ ;           • changed line
5:    $\mathcal{F} \leftarrow \mathcal{N}^*(\mathbf{i})$ ;
6:    $\mathcal{B} \leftarrow \mathcal{P}^*(\mathbf{i}) \cap \mathcal{S}$ ;           • changed for Kronecker
7:    $\mathcal{S}' \leftarrow \mathcal{S}' \setminus \mathcal{B}$ ;
8:   if  $\mathcal{F} \subseteq \mathcal{B}$  then
9:      $C \leftarrow C + 1$ ;
10:     $\mathcal{R}_C \leftarrow \mathcal{F}$ ;
11:     $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B} \setminus \mathcal{F}$ ;
12:     $\mathcal{D} \leftarrow \text{IntersectionMax}(\mathcal{D}, \mathcal{B} \cdot \infty)$ ;   • new line
13:   else
14:      $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{B}$ ;
15:     $\mathcal{D} \leftarrow \text{ComputeDistances}(\mathcal{S} \setminus \mathcal{S}')$ ; • new line
16:   end if
17: end while

```

Figure 7. Heuristic-based classification

in Figure 7 in procedure Distance1Classify. Differences from procedure ReachClassify are indicated. We assume that the EV⁺MDD encoding the state distances from the initial states $\mathcal{S}^0$ has already been constructed, as described in [9]. Note that Distance1Classify is nearly identical to ReachClassify, except that seed states are selected using our distance heuristic (via SelectFurthest), instead of randomly. Also, we intersect the backward set with the set $\mathcal{S}$ to remove unreachable states, as discussed earlier.

At the end of each iteration, the distance EV⁺MDD is modified by setting distances of infinity for states that have already been classified (line 15). This can be done by first constructing an EV⁺MDD representing the function $b = \mathcal{B} \cdot \infty$, where $b(\mathbf{i}) = \infty$ if $\mathbf{i} \in \mathcal{B}$ and $b(\mathbf{i}) = 0$ if $\mathbf{i} \notin \mathcal{B}$; this can be done efficiently using the MDD representing $\mathcal{B}$: MDD paths leading to terminal 0 become EV⁺MDD

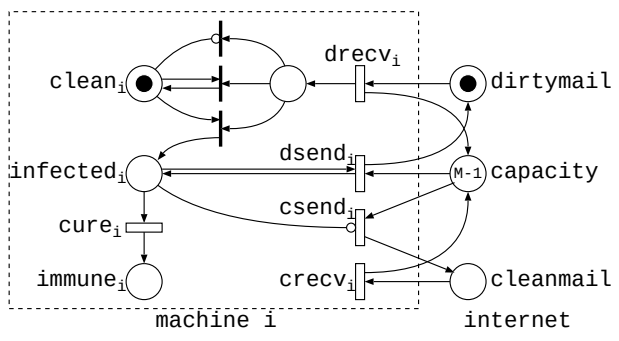
paths with edge-values summing to 0, and MDD paths leading to terminal 1 become EV⁺MDD paths containing an infinite edge value. The distance function $\mathcal{D}$ is then intersected with function b . Thus, the additional computational cost of using procedure Distance1Classify instead of procedure ReachClassify is *a priori* construction of the distance function $\mathcal{D}$ and an update to $\mathcal{D}$ for each iteration.

A classification algorithm based on heuristic (2) is also shown in Figure 7, in procedure Distance2Classify. Like Distance1Classify, seed states are selected via SelectFurthest. When the seed state is recurrent, the distance EV⁺MDD is updated so that the classified states have distance of infinity, as done in Distance1Classify. When the seed state is transient, the old distance EV⁺MDD is destroyed. Distances from the classified states are then reconstructed using procedure ComputeDistances (not shown), as described in [9]. Thus, if Distance2Classify requires n iterations, then the distance function $\mathcal{D}$ is constructed exactly $C - n + 1$ times. When $n = C$, Distance2Classify requires the same computational overhead (compared to ReachClassify) as Distance1Classify.

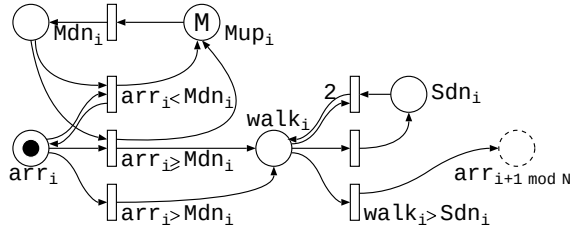
4. Experimental Results

Prototypes of our techniques are implemented in the tool SMART [6]. We report results for several Markov chains, described using stochastic Petri nets shown in Figure 8 and discussed in detail below. Experiments were performed using 933 Mhz Pentium III machines running Linux. For each model, we report the number of reachable states $|\mathcal{S}|$ in the Markov chain, the number of transient states $|\mathcal{T}|$, the number of recurrent classes C , and performance measures for algorithms ReachClassify, Distance1Classify, and Distance2Classify. For all algorithms, we use saturation-based methods to construct forward and backward reachability sets. In practice, absorbing states of a Markov chain could be detected and classified in a preprocessing step, using a simple query [20]; we do not do so here.

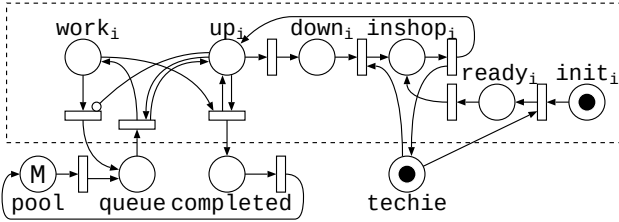
Performance measures we consider for the three methods include the number of iterations required, under columns “#iters”. We also report the total CPU time for classification; for Distance1Classify and Distance2Classify, this includes the time required to construct the initial distance function $\mathcal{D}$. Finally, we report the peak number of nodes required during construction, as a rough indication of the memory required (other structures, such as operation caches, can also contribute to the memory requirements, but these are usually related to the number of nodes). For all algorithms, we report the peak number of MDD nodes required under columns “MDD”. For Distance1Classify and Distance2Classify, we also report the peak number of EV⁺MDD nodes required for distance function computations, under columns “ $\mathcal{D}$ ”.



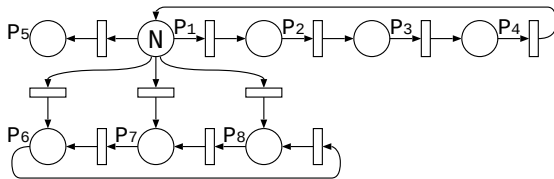
(a) Internet virus model



(b) Multiserver polling system with failing servers



(c) Web server model



(d) Model with nearby recurrent classes

Figure 8. Models used in experiments

4.1. Virus model

The first model we consider, shown in Figure 8(a), is a simple model of a computer virus that spreads via email. When a clean machine (i.e., not yet infected) receives a message containing the virus, it is possible that nothing happens or the machine can become infected. Eventually, when system administrators discover that the machine has been infected and update the anti-virus software, the machine becomes immune. An infected machine sends additional messages containing the virus, while clean and immune machines send virus-free messages. An infected or immune

machine is not affected by messages containing the virus. There is a finite capacity for email in transit, represented by parameter M ; thus, corrupt messages and intact messages compete for bandwidth. The portion of the model enclosed in the dashed rectangle is replicated N times to represent N machines. The virus model stabilizes once no machine is infected and no messages containing the virus are in transit. After this happens, any machine can be either clean or immune; thus, the system has 2^N recurrent classes, each with $M + 1$ states. It can be shown that the total number of (tangible) reachable states for the initial state shown is $(3^N - 1)(M + 2)(M + 1)/2$.

Results for the virus model are shown in Table 1. Note that, for the parameters N and M that we consider, the number of recurrent states is a small fraction of the total number of reachable states. This implies that, with probability close to one, the first seed state selected by ReachClassify will be transient. If a small number of transient states are classified at each iteration on average, then the expected number of iterations using random selection will be large. On the other hand, the number of iterations can still be small, even though most states are transient, if a large number of transient states are classified during each iteration. For this particular model, we see that the number of iterations required tends to be a bit larger than $2C$: twice the smallest possible number. This suggests that a large number of transient states are classified at each iteration. Note that, as N is fixed and M increases (thereby decreasing the fraction of recurrent states), the number of iterations tends to increase.

For this model, we can see that both distance heuristics perform quite well: the number of iterations for all cases we checked was $C + 1$, which means that a transient seed state was selected only once. As a result, the CPU times for Distance1Classify and Distance2Classify are quite low. Interestingly, while the number of iterations is about half that of ReachClassify, notice that the total CPU time is much less than half. Finally, we note that use of the distance heuristics allowed the analysis of larger models (the dashes correspond to cases that could not complete due to excessive memory requirements).

4.2. Polling model

The second model, shown in Figure 8(b), is a multiserver polling system similar to [2]: there are N sites (a single site is shown in the figure), each containing M machines that can fail. Servers (initially one per site) move from site to site, repairing the failed machines. However, unlike [2], we assume that the servers are also machines that can fail; for simplicity, this can occur only while a server is in transit from one site to another. A working server will attempt to repair a failed server if it encounters one; however, it is possible that the repairing server can also fail. This model de-

N	M	$ S $	$ T $	C	ReachClassify			Distance1Classify				Distance2Classify			
					#iters	CPU	MDD	#iters	CPU	MDD	$\mathcal{D}$	#iters	CPU	MDD	$\mathcal{D}$
8	10	4.33×10^5	4.30×10^5	256	559	37.4	9,650	257	2.9	5,949	1,087	257	2.9	5,956	13,823
8	20	1.52×10^6	1.51×10^6	256	572	96.5	10,921	257	4.4	5,949	1,087	257	5.8	5,956	54,068
8	30	3.25×10^6	3.25×10^6	256	661	302.3	8,780	257	6.7	5,949	1,087	257	10.7	5,956	120,813
8	40	5.65×10^6	5.64×10^6	256	667	694.5	8,867	257	10.2	5,949	1,087	257	18.0	5,956	214,058
8	50	8.70×10^6	8.69×10^6	256	676	1320.5	8,881	257	14.1	5,949	1,087	257	28.5	5,956	333,803
10	10	3.90×10^6	3.89×10^6	1,024	—	—	—	1,025	16.7	11,314	1,987	1,025	13.8	11,323	21,372
10	12	5.37×10^6	5.36×10^6	1,024	—	—	—	1,025	17.8	11,314	1,989	1,025	15.1	11,323	30,557
10	14	7.09×10^6	7.07×10^6	1,024	—	—	—	1,025	18.9	11,314	1,989	1,025	16.6	11,323	41,386
10	16	9.03×10^6	9.02×10^6	1,024	—	—	—	1,025	20.8	11,314	1,989	1,025	18.2	11,323	53,859
12	12	4.84×10^7	4.83×10^7	4,096	—	—	—	4,097	107.0	19,207	3,285	4,097	80.7	19,218	43,718

Table 1. Results for the virus model

scribes an absorbing Markov chain, where absorbing states correspond to cases where all servers and all machines have failed. Since any of the N servers can fail at any of the N sites, the number of absorbing states is exactly $\binom{2N-1}{N}$, which depends only on parameter N .

Results for the polling model are shown in Table 2. In this case, the number of iterations required for random selection is quite large, about two orders of magnitude more than the smallest possible number, and increases rapidly with M . In this case, the distance heuristic reduces the number of iterations to C ; thus, Distance1Classify and Distance2Classify are identical. Both distance heuristics drastically reduce the CPU time required. Again, the distance heuristics allow for the analysis of much larger models.

4.3. Web server model

The third model, shown in Figure 8(c), represents a simple cluster of web servers. A single server is represented by the portion enclosed in the dashed rectangle; this is replicated N times. Users submit service requests via the Internet (at most M are allowed within the system at a time), and these requests are added to a queue. Queued requests are assigned to one of the N servers; when the request is completed, results are returned to the user. Servers may fail, and are repaired by a technician. In addition, we assume that the system is not quite ready for production: each server requires some initialization by the technician before it can process requests. This model describes a Markov chain with a single recurrent class. However, there are several transient states due to the initialization of the servers.

Results for the web server model are shown in Table 3. For this model, it is easy to select a seed state that results in a large number of states classified: if the seed state has the set of servers $\mathcal{I}$ initialized and ready to process requests, then the backward set from the seed state includes all states where the set of initialized servers is a subset of $\mathcal{I}$. Note that the recurrent class corresponds to the case $\mathcal{I} = \{1, \dots, N\}$. We find that random selection of seed states works quite

well: a few iterations are enough to classify the states. The distance heuristics achieve the smallest possible number of iterations in this case (and thus are identical), and reduce the total CPU time required.

4.4. Nearby recurrent class model

The fourth model, shown in Figure 8(d), is designed to defeat our distance heuristic. The system stabilizes when there are no tokens in places P_1, P_2, P_3, P_4 . There are $N+1$ recurrent classes, each with a different number of tokens in place P_5 . Each recurrent class can be reached in exactly N steps from the initial state. However, there are many transient states with longer distance from the initial state; in particular, the state with N tokens in place P_4 is $3N$ steps from the initial state. Indeed, it can be shown that there are $(N+1)(N+2)/2$ SCCs (with $N+1$ terminal SCCs), and Distance1Classify will visit each SCC once.

Results for the nearby recurrent class model are shown in Table 4. As expected, Distance1Classify performs quite poorly for this model, significantly worse than ReachClassify. However, we see that Distance2Classify performs much better than Distance1Classify, and is competitive with ReachClassify. Indeed, Distance2Classify requires only $C+1$ iterations: after selecting the first transient seed state (with distance $3N$ from the initial state), the distance function is reconstructed, and from then on only recurrent seed states are selected. Unfortunately, this savings in number of iterations does not translate into CPU savings with respect to ReachClassify; we are currently investigating whether this is due to distance function overhead, or to the selected seed states.

4.5. Closed queuing network

We also consider a fifth model, not shown, which is similar to the closed queuing network model of [24], except that we use a continuous-time Markov chain. Thus, we obtain

N	M	$ S $	$ T $	C	ReachClassify			Distance1Classify				Distance2Classify			
					#iters	CPU	MDD	#iters	CPU	MDD	$\mathcal{D}$	#iters	CPU	MDD	$\mathcal{D}$
4	4	8.53×10^5	8.53×10^5	35	1,550	89.0	1,559	35	0.7	278	31	35	0.7	279	31
4	5	1.77×10^7	1.77×10^7	35	2,394	224.6	2,277	35	0.9	298	31	35	0.9	299	31
4	6	3.28×10^6	3.28×10^6	35	3,445	500.1	2,937	35	1.2	318	31	35	1.2	319	31
4	10	2.00×10^7	2.00×10^7	35	—	—	—	35	2.7	398	31	35	2.7	399	31
4	20	2.65×10^8	2.65×10^8	35	—	—	—	35	8.3	598	31	35	8.4	599	31
5	5	9.04×10^7	9.04×10^7	126	—	—	—	126	7.8	709	62	126	7.7	710	62
6	6	1.19×10^{10}	1.19×10^{10}	462	—	—	—	462	85.5	1,902	112	462	85.7	1,903	112
7	7	1.86×10^{12}	1.86×10^{12}	1,716	—	—	—	1,716	918.3	5,710	188	1,716	916.6	5,711	188

Table 2. Results for the polling model

N	M	$ S $	$ T $	C	ReachClassify			Distance1Classify				Distance2Classify			
					#iters	CPU	MDD	#iters	CPU	MDD	$\mathcal{D}$	#iters	CPU	MDD	$\mathcal{D}$
10	10	2.02×10^9	1.89×10^9	1	2	21.8	212,112	1	17.4	159,755	46,157	1	18.2	159,755	46,157
10	12	3.19×10^9	2.96×10^9	1	2	27.1	265,866	1	18.1	162,829	47,609	1	18.8	162,829	47,609
12	12	7.94×10^{10}	7.59×10^{10}	1	5	140.3	788,826	1	64.5	422,196	104,961	1	66.5	422,196	104,961
12	14	1.18×10^{11}	1.13×10^{11}	1	8	179.7	898,874	1	68.0	426,996	106,929	1	68.0	426,996	106,929
14	14	2.93×10^{12}	2.85×10^{12}	1	—	—	—	1	236.3	984,801	212,972	1	243.0	984,801	212,972

Table 3. Results for the web server model

the same number of recurrent classes, but our total number of reachable states differs slightly from [24]. Results for the closed queuing network model are shown in Table 5. Since the number of transient states is a small fraction of the overall number of states, we expect that random selection of seed states will usually select a recurrent state. Indeed, we see that the number of iterations required by ReachClassify is very close to C . Both distance heuristics use the minimum number of iterations. In this case, the computational overhead required to manage the distance functions is quite small, and so Distance1Classify and Distance2Classify perform slightly better than ReachClassify.

5. Conclusion

Current state-of-the-art implicit methods to classify the states of a Markov chain into transient and recurrent classes use an iterative approach, in which a seed state is selected at the start of each iteration. To reduce the total number of iterations required to the minimum possible, it is necessary that the seed state selected is always recurrent. This is usually not achieved using random seed selection, especially when the number of transient states is significantly larger than the number of recurrent states.

We presented two heuristics that attempt to select recurrent states based on state distance. The first heuristic is to select a seed state from the unclassified states with furthest distance from the initial state of the system. The second heuristic is similar, except that the distance function is updated whenever a transient state is selected. Efficient im-

plementation of the distance heuristics is possible thanks to EV⁺MDDs. Experimentally, we find that the two heuristics are quite successful, often reducing the number of iterations required to the minimum possible. The heuristics often allowed for the analysis of larger Markov chains than was possible using random selection of seed states.

We presented examples for which the second heuristic performs significantly better than the first heuristic. Further study is needed to determine if the reverse is also possible.

References

- [1] A. Aho, J. Hopcroft, and J. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [2] M. Ajmone Marsan, S. Donatelli, F. Neri, and A. Scalia. Approximate GSPN analysis of multiserver polling systems. In *Proceedings of the International Conference on Telecommunication, Distribution and Parallelism (TDP'96)*, La Londe Les Maures, France, June 1996.
- [3] R. Bloem, H. Gabow, and F. Somenzi. An algorithm for strongly connected component analysis in $n \log n$ symbolic steps. In *Proc. International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pages 37–54. Springer-Verlag, Nov. 2000.
- [4] R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.*, C-35(8):677–691, Aug. 1986.
- [5] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.*, 12(3):203–222, SUMMER 2000.

N	$ S $	$ T $	C	ReachClassify			Distance1Classify				Distance2Classify			
				#iters	CPU	MDD	#iters	CPU	MDD	$\mathcal{D}$	#iters	CPU	MDD	$\mathcal{D}$
20	8.88×10^5	8.86×10^5	21	115	6.8	3,566	231	20.5	4,128	1,379	22	4.9	5,020	48,696
30	1.03×10^7	1.03×10^7	31	180	21.7	9,043	496	110.7	8,036	2,079	32	21.6	10,520	50,868
40	6.29×10^7	6.29×10^7	41	222	43.4	13,251	861	394.2	13,196	2,779	42	66.6	18,020	47,701
50	2.64×10^8	2.64×10^8	51	336	103.0	17,390	–	–	–	–	52	167.3	27,520	47,500

Table 4. Results for the nearby recurrent class model

N	$ S $	$ T $	C	ReachClassify			Distance1Classify				Distance2Classify			
				#iters	CPU	MDD	#iters	CPU	MDD	$\mathcal{D}$	#iters	CPU	MDD	$\mathcal{D}$
10	2.41×10^9	1.02×10^3	11	12	18.9	87,603	11	16.8	74,416	24,021	11	17.4	74,416	24,021
12	2.14×10^{10}	4.10×10^3	13	14	63.6	205,081	13	52.2	177,327	56,718	13	53.8	177,327	56,718
14	1.85×10^{13}	1.64×10^4	15	16	191.9	499,272	15	146.4	369,871	119,211	15	150.6	369,871	119,211
16	1.57×10^{15}	6.55×10^4	17	–	–	–	17	402.4	700,299	229,970	17	398.8	700,299	229,970

Table 5. Results for the closed queuing network

- [6] G. Ciardo, R. Jones, A. Miner, and R. Siminiceanu. Logical and stochastic modeling with SMART. In P. Kemper and W. H. Sanders, editors, *Proc. 13th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, LNCS 2794, pages 78–97, Urbana, IL, USA, Sept. 2003. Springer-Verlag.
- [7] G. Ciardo, G. Luetgen, and R. Siminiceanu. Saturation: an efficient iteration strategy for symbolic state space generation. In T. Margaria and W. Yi, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2031, pages 328–342, Genova, Italy, Apr. 2001. Springer-Verlag.
- [8] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. In P. Buchholz, editor, *8th Int. Workshop on Petri Nets and Performance Models (PNPM’99)*, pages 22–31, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [9] G. Ciardo and R. Siminiceanu. Using edge-valued decision diagrams for symbolic generation of shortest paths. In M. D. Aagaard and J. W. O’Leary, editors, *Proc. Fourth International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, LNCS 2517, pages 256–273, Portland, OR, USA, Nov. 2002. Springer-Verlag.
- [10] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.
- [11] P. Fernandes, B. Plateau, and W. J. Stewart. Efficient descriptor-vector multiplication in stochastic automata networks. *J. ACM*, 45(3):381–414, 1998.
- [12] G. Hachtel, E. Macii, A. Pardo, and F. Somenzi. Markovian analysis of large finite state machines. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, 15(12):1479–1493, Dec. 1996.
- [13] H. Hermanns, J. Kayser, and M. Siegle. Multi terminal binary decision diagrams to represent and analyse continuous time Markov chains. In B. Plateau, W. J. Stewart, and M. Silva, editors, *Numerical Solution of Markov Chains*, pages 188–207, Zaragoza, Spain, June 1999. Prensas Universitarias de Zaragoza.
- [14] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [15] J. G. Kemeny and J. L. Snell. *Finite Markov Chains*. D. Van Nostrand Company, Inc., 1960.
- [16] P. Kemper. Numerical analysis of superposed GSPNs. *IEEE Trans. Softw. Eng.*, 22(9):615–628, Sept. 1996.
- [17] S. Kimura and E. M. Clarke. A parallel algorithm for constructing binary decision diagrams. In *Proc. Int. Conf. on Computer Design (ICCD)*, pages 220–223, Cambridge, MA, Sept. 1990. IEEE Comp. Soc. Press.
- [18] Y. Matsunaga, P. C. McGeer, and R. K. Brayton. On computing the transitive closure of a state transition relation. In *Proc. 30th International Design Automation Conference*, pages 260–265. ACM Press, 1993.
- [19] A. S. Miner. Implicit GSPN reachability set generation using decision diagrams. *Perf. Eval.*, 56(1-4):145–165, Mar. 2004.
- [20] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 6–25, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [21] B. Plateau. On the stochastic structure of parallelism and synchronisation models for distributed algorithms. In *Proc. 1985 ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 147–153, Austin, TX, USA, May 1985.
- [22] W. J. Stewart. *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [23] H. J. Touati, R. K. Brayton, and R. Kurshan. Testing language containment for ω -automata using BDDs. *Information and Computation*, 118(1):101–109, 1995.
- [24] A. Xie and P. A. Beerel. Efficient state classification of finite state Markov chains. *IEEE Trans. Computer-Aided Design*, 17(12):1334–1339, 1998.